AI_ASSIST CODING LAB ASSIGNMENT -16.2

Program Name: B. Tech
Academic Year: 2025-2026

Course Code: 23CS002PC304
Course Title: AI Assisted Coding
Year/Sem: III / II

Student Name: Ravula seetharam reddy
Roll No: 2303A52440
Batch: 34

Assignment Number: 16.2 / 24

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

---

Lab Objectives

- Understand schema design principles for relational databases.
- Use AI-assisted tools to generate SQL schema definitions.
- Write and refine SQL queries (CRUD + aggregate operations).
- Explore how AI can optimize queries and ensure normalization.

Task 1 – Design Database Schema for Student Management System

Prompt Used

Generate SQL CREATE TABLE statements for Students, Courses and Enrollments tables with primary keys and foreign keys following normalization rules.

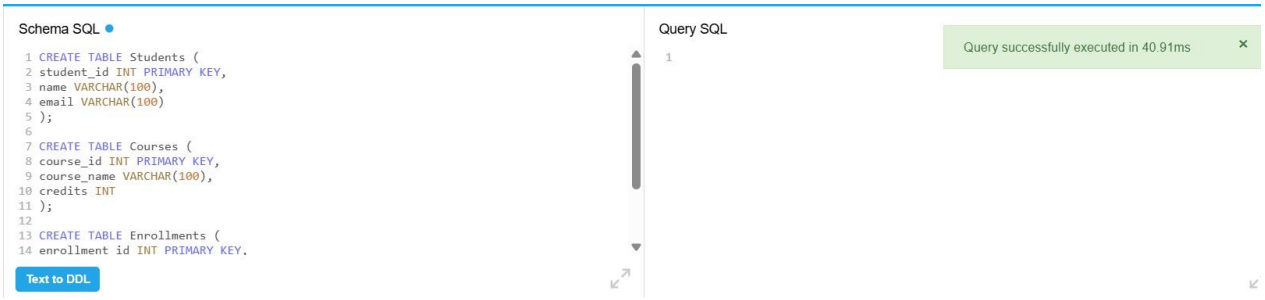
SQL Code

```sql
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);

CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100),
credits INT
);

CREATE TABLE Enrollments (
enrollment_id INT PRIMARY KEY,
student_id INT,
course_id INT,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Output

Tables Students, Courses and Enrollments created successfully.

Screenshot of Execution – Task 1



Task 2 – Insert Sample Records

Prompt Used

Generate INSERT SQL statements to populate Students, Courses and Enrollments tables.

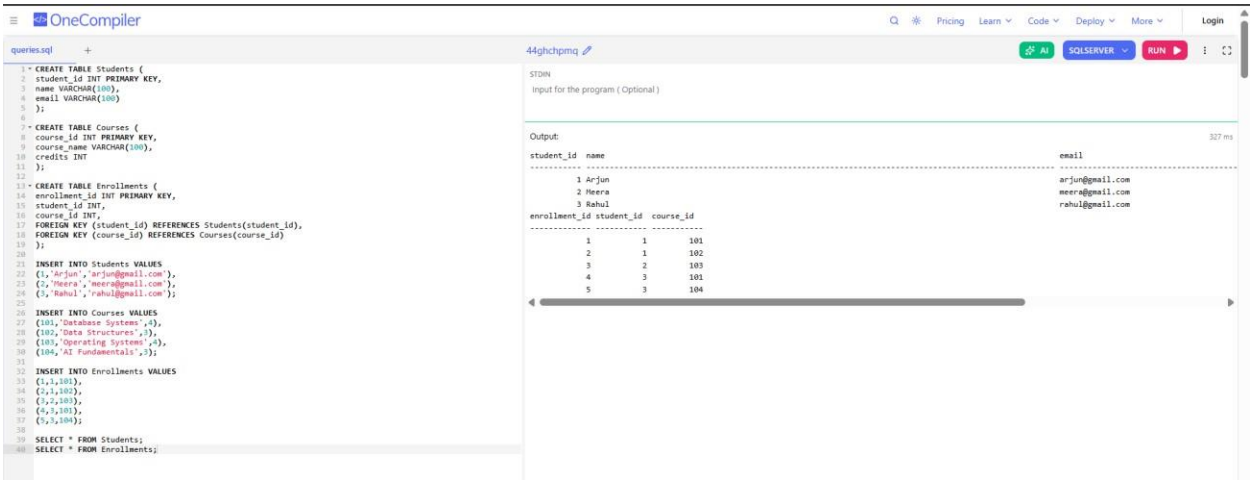
SQL Code

```sql
INSERT INTO Students VALUES
(1,'Arjun','arjun@gmail.com'),
(2,'Meera','meera@gmail.com'),
(3,'Rahul','rahul@gmail.com');

INSERT INTO Courses VALUES
(101,'Database Systems',4),
(102,'Data Structures',3),
(103,'Operating Systems',4),
(104,'AI Fundamentals',3);

INSERT INTO Enrollments VALUES
(1,1,101),
(2,1,102),
(3,2,103),
(4,3,101),
(5,3,104);

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output

Student and enrollment records displayed successfully.

Screenshot of Execution – Task 2



Task 3 – Perform CRUD Operations

Prompt Used

Generate SQL queries for retrieving courses, updating student email and deleting an enrollment record.

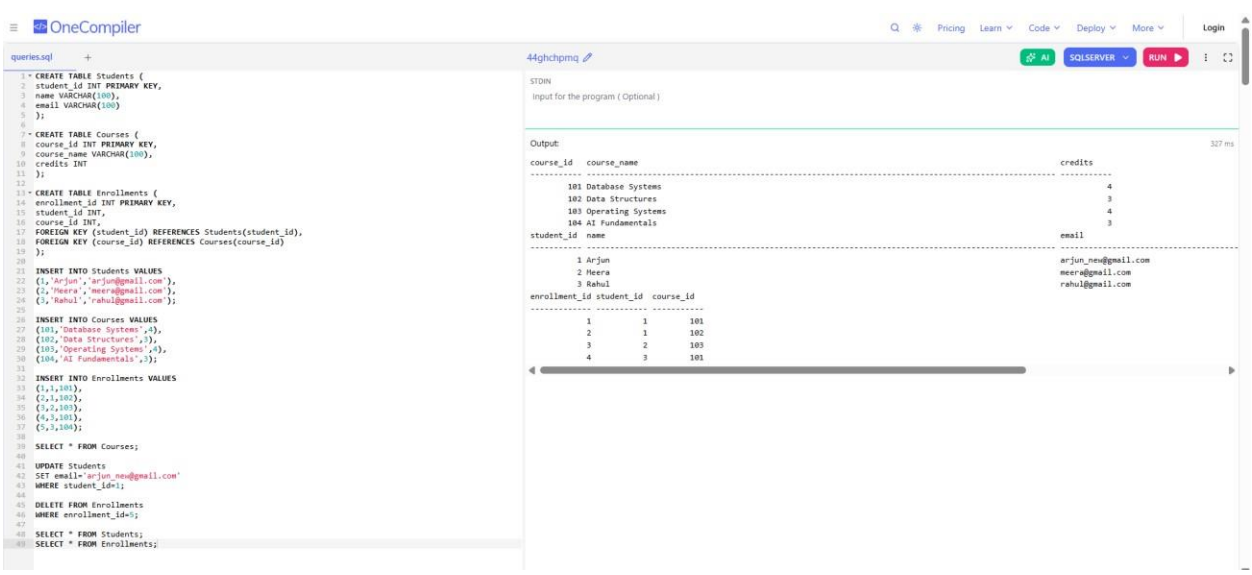
SQL Code

```sql
SELECT * FROM Courses;

UPDATE Students
SET email='arjun_new@gmail.com'
WHERE student_id=1;

DELETE FROM Enrollments
WHERE enrollment_id=5;

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output

Records updated and deleted successfully.

Screenshot of Execution – Task 3



Task 4 – Execute Aggregate Queries

Prompt Used

Generate SQL queries to count number of students per course and list courses having more than one student.

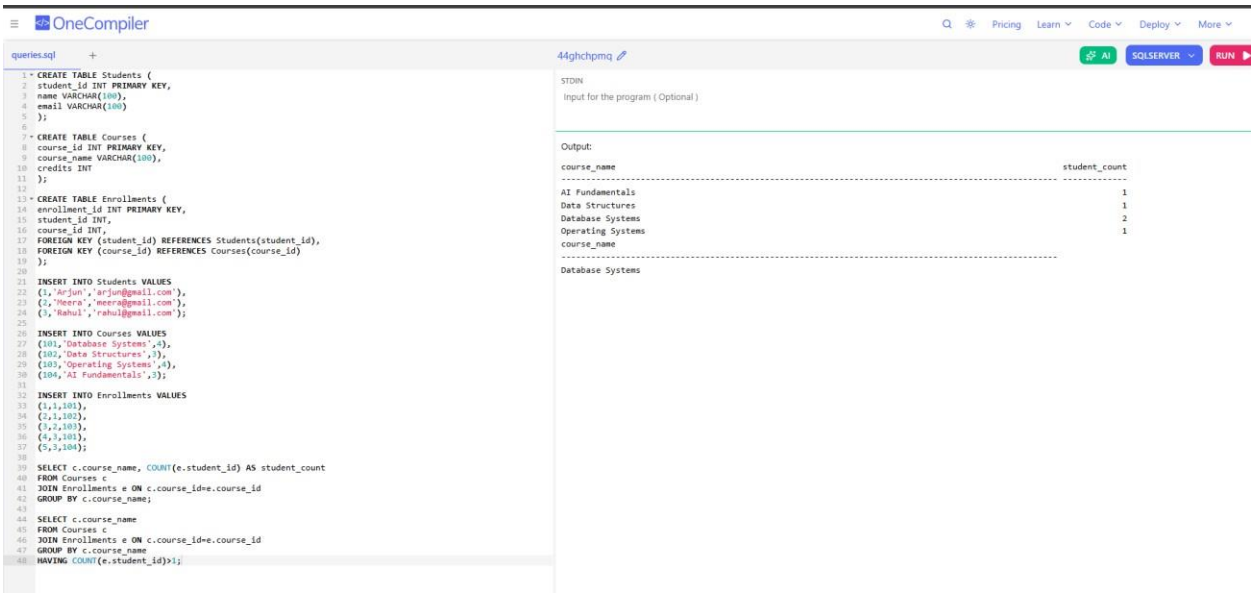
SQL Code

```sql
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id)>1;
```

Output

Student counts per course and courses with multiple enrollments displayed.

Screenshot of Execution – Task 4



Task 5 – Query Optimization

Prompt Used

Compare subquery and JOIN based query to retrieve students enrolled in course 101.

SQL Code

```sql
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id=101
);

-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id=e.student_id
WHERE e.course_id=101;
```

Output

Both queries return the same students enrolled in course 101.

Screenshot of Execution – Task 5

OneCompiler

queries.sql

```sql
=CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);

CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100),
credits INT
);

CREATE TABLE Enrollments (
enrollment_id INT PRIMARY KEY,
student_id INT,
course_id INT,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

INSERT INTO Students VALUES
(1,'Arjun','arjun@gmail.com'),
(2,'Meera','meera@gmail.com'),
(3,'Rahul','rahul@gmail.com');

INSERT INTO Courses VALUES
(101,'Database Systems',4),
(102,'Data Structures',3),
(103,'Operating Systems',4),
(104,'AI Fundamentals',3);

INSERT INTO Enrollments VALUES
(1,1,101),
(2,1,102),
(3,2,103),
(4,3,101),
(5,3,104);

SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id=101
);

SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id=e.student_id
WHERE e.course_id=101;
```

44ghchpmq

STDIN

Input for the program (Optional)

Output:

```
Msg 102, Level 15, State 1, Server 99a18edb38c1, Line 2
Incorrect syntax near '='.
```

Conclusion

This lab demonstrated relational database schema design, insertion of records, CRUD operations, aggregate queries and SQL query optimization using JOIN operations.